Examination Result and Grace-Mark Validation

1. Problem Statement

To write a Python program to display a student's grade based on the marks obtained.

2. Objectives

- To understand if-elif-else statements.
- To classify marks into different grades.

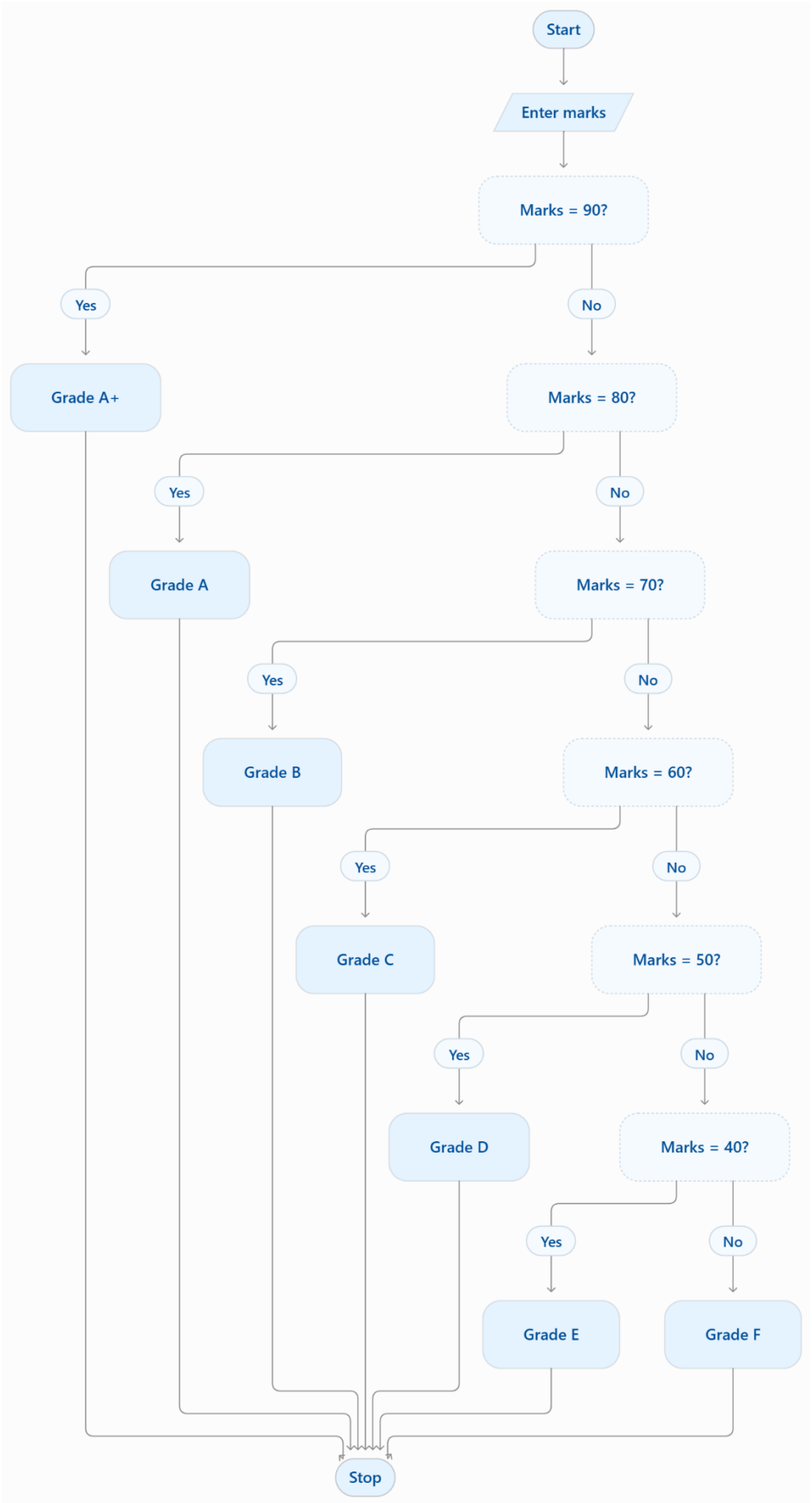
3. Assumptions

- Marks are entered as an integer.
- Marks range from 0 to 100.
- Grace marks are not calculated in this program; it only displays grades.

4. Algorithm

1. Start.
2. Read the student's marks.
3. Check whether marks are between 0 and 100.
4. If marks are 90 or above, display A+ grade.
5. Otherwise, check the remaining grade conditions.
6. Display the appropriate grade.
7. Stop.

5. Flowchart


Start
Enter marks
Marks = 90?
Yes
No
Grade A+
Marks = 80?
Yes
No
Grade A
Marks = 70?
Yes
No
Grade B
Marks = 60?
Yes
No
Grade C
Marks = 50?
Yes
No
Grade D
Marks = 40?
Yes
No
Grade E
Grade F
Stop

6. Pseudocode

```
START
INPUT marks
IF marks < 0 OR marks > 100
   PRINT "Invalid marks"
ELSE IF marks >= 90
   PRINT "A+ grade"
ELSE IF marks >= 80
   PRINT "A grade"
ELSE IF marks >= 70
   PRINT "B grade"
ELSE IF marks >= 60
   PRINT "C grade"
ELSE IF marks >= 50
   PRINT "D grade"
ELSE IF marks >= 40
   PRINT "E grade"
ELSE
   PRINT "F grade"
STOP
```

7. Python Source Code

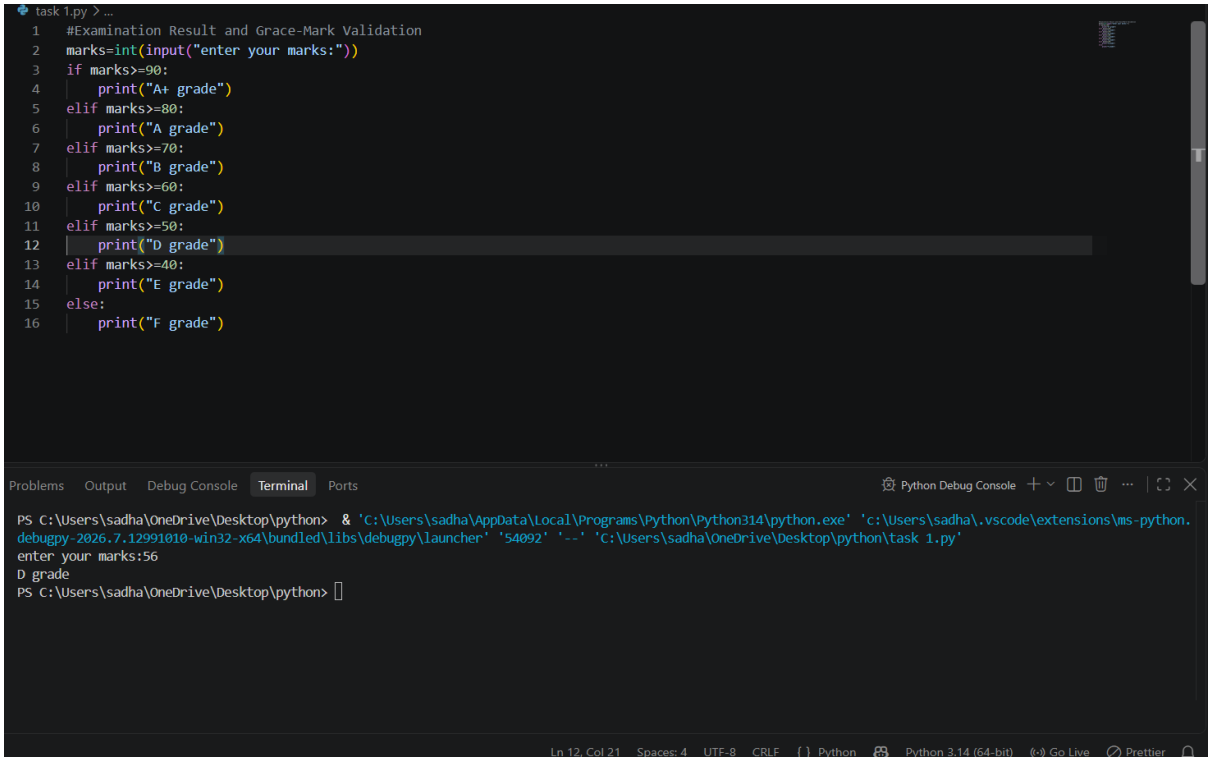
```
marks = int(input("Enter your marks: "))
if marks < 0 or marks > 100:
  print("Invalid marks")
elif marks >= 90:
   print("A+ grade")
elif marks >= 80:
```

```
 print("A grade")
elif marks >= 70:
  print("B grade")
elif marks >= 60:
  print("C grade")
elif marks >= 50:
  print("D grade")
elif marks >= 40:
  print("E grade")
else:
 print("F grade")
```

8. Test Cases

Input	**Expected output**
95	A+ grade
85	A grade
72	B grade
35	F grade
105	Invalid marks

9.sample output



10. Conclusion

The program successfully classifies a student's marks into grades using conditional statements.

Program 2: Attendance Recovery Computation

1. Problem Statement

To calculate a student's attendance percentage and find the extra classes needed to reach 75% attendance.

2. Objectives

- To calculate attendance percentage.
- To use if-else and while loops.

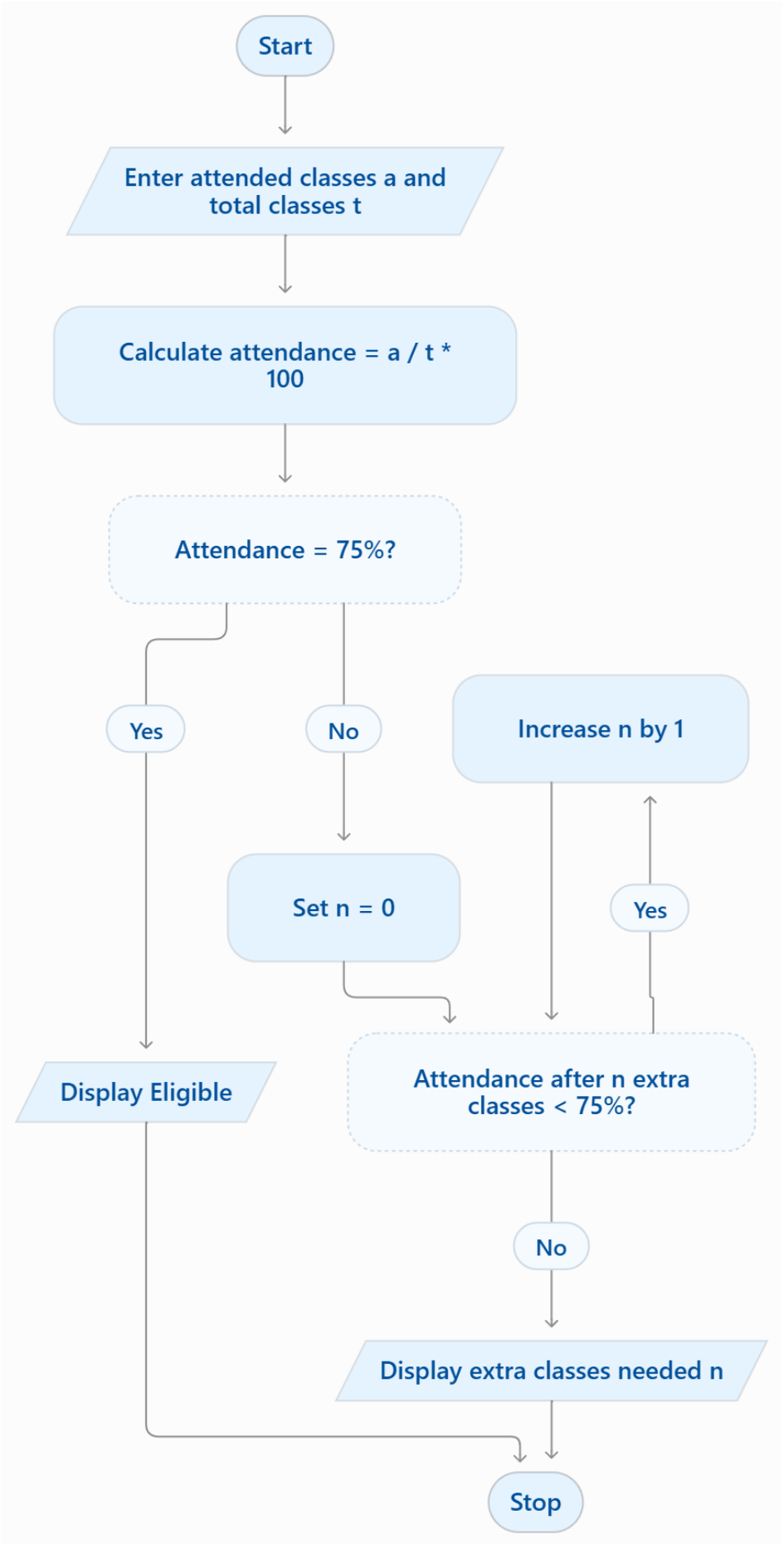
3. Assumptions

- Total classes must be greater than zero.
- Attended classes must be between zero and total classes.
- The required attendance is 75%.

4. Algorithm

1. Start.
2. Input classes attended and total classes.
3. Validate the inputs.
4. Calculate attendance percentage.
5. If attendance is at least 75%, display Eligible.
6. Otherwise, calculate the extra classes needed using a loop.
7. Display the result.
8. Stop.

5. Flowchart


Start
Enter attended classes a and total classes t
Calculate attendance = a / t * 100
Attendance = 75%?
Yes
No
Increase n by 1
Set n = 0
Yes
Display Eligible
Attendance after n extra classes < 75%?
No
Display extra classes needed n
Stop

6. Pseudocode

```
START
INPUT attended, total
IF total <= 0 OR attended < 0 OR attended > total
   PRINT "Invalid input"
ELSE
   percentage = attended / total * 100
   PRINT percentage
   IF percentage >= 75
      PRINT "Eligible"
   ELSE
      n = 0
      WHILE (attended + n) / (total + n) * 100 < 75
         n = n + 1
      PRINT n
STOP
```

7. Python Source Code

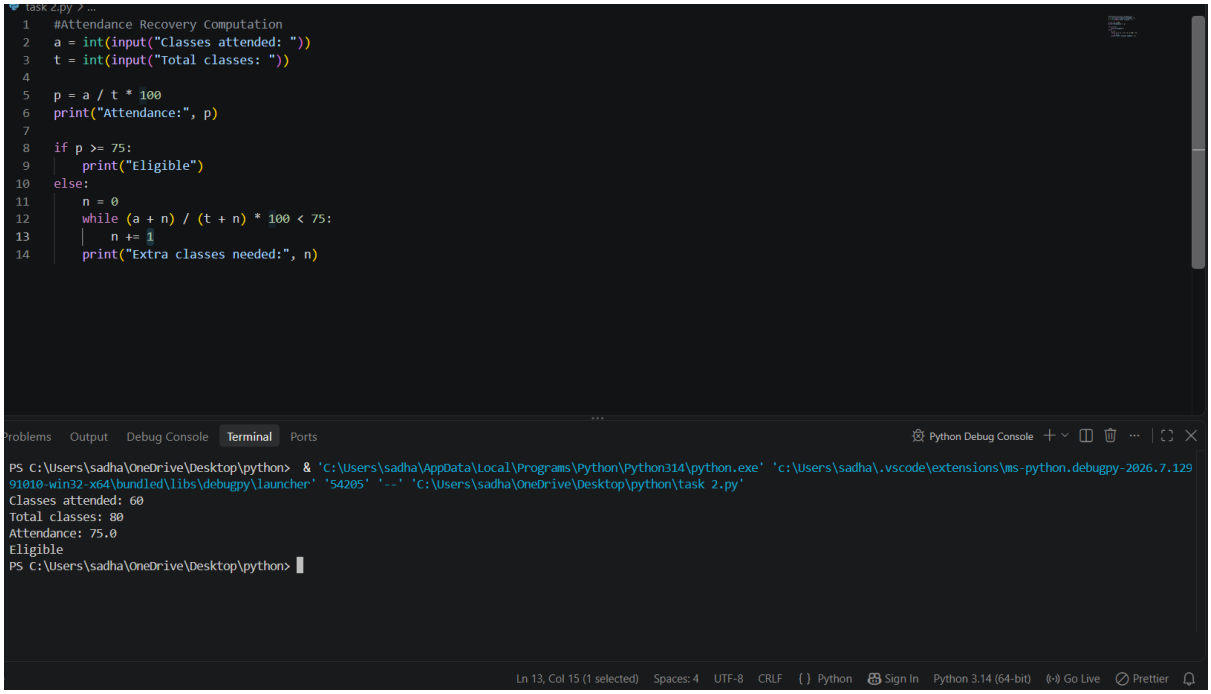
```
a = int(input("Classes attended: "))
t = int(input("Total classes: "))
if t <= 0 or a < 0 or a > t:
  print("Invalid input")
else:    p = a / t * 100
 print("Attendance:", round(p, 2), "%")
  if p >= 75:
  print("Eligible")
 else:
     n = 0
```

```
while (a + n) / (t + n) * 100 < 75:
    n += 1
print("Extra classes needed:", n)
```

8. Test Cases

Attended	Total	Expected output
8	10	80%; Eligible
6	10	60%; 6 extra classes
0	0	Invalid input
5	5	100%; Eligible

9. Output



10. Conclusion

The program calculates attendance percentage and determines the extra classes needed to reach the required attendance.

Program 3: ATM Transaction Sequence Simulation

1. Problem Statement

To simulate ATM deposit and withdrawal transactions and display the final account balance.

2. Objectives

- To understand for loops and conditional statements.
- To perform deposit and withdrawal operations.

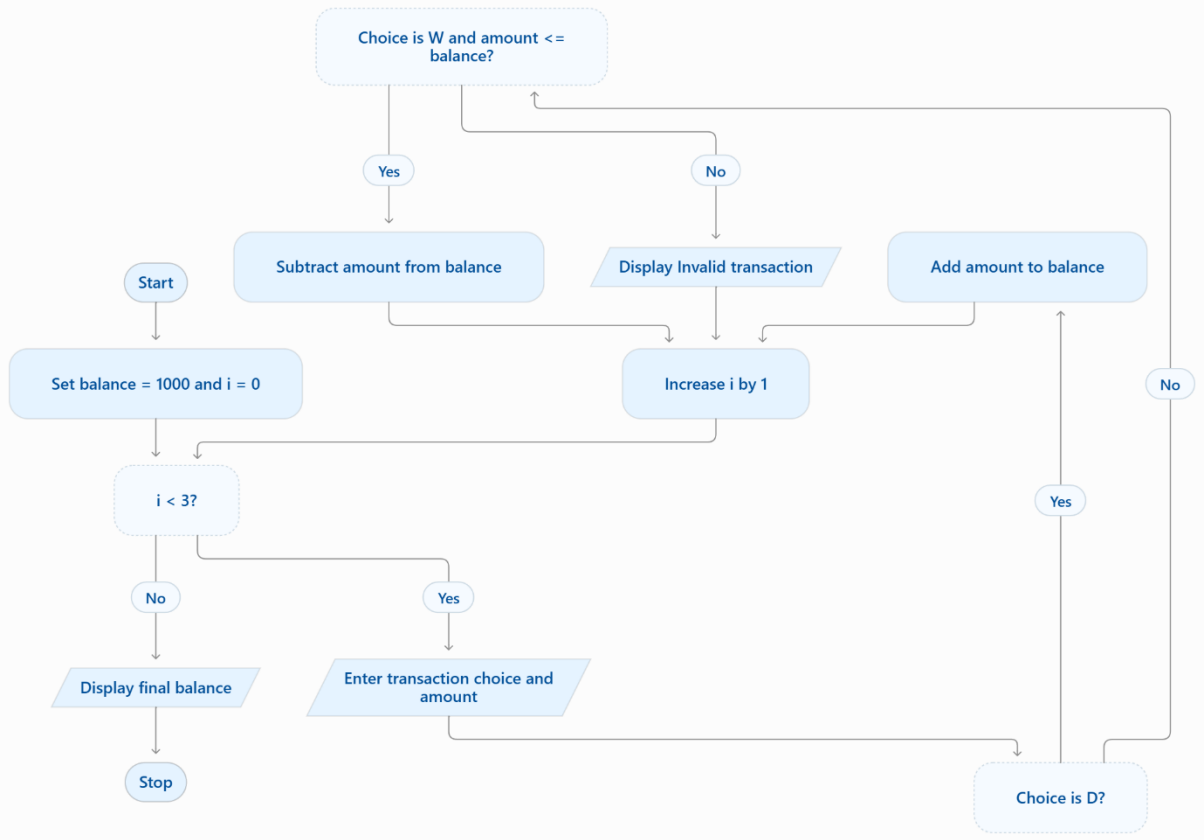
3. Assumptions

- Initial balance is ₹1000.
- Exactly three transactions are performed.
- Transaction amounts must be positive.

4. Algorithm

1. Start.
2. Set balance to 1000.
3. Repeat three times.
4. Input transaction type and amount.
5. If deposit, add the amount to balance.
6. If withdrawal and sufficient balance, subtract the amount.
7. Otherwise, display an invalid transaction message.
8. Display final balance and stop.

5. Flowchart



6. Pseudocode

```
START
balance = 1000
FOR i = 1 TO 3
   INPUT choice, amount
   IF amount <= 0
      PRINT "Invalid amount"
   ELSE IF choice = "D"
      balance = balance + amount
   ELSE IF choice = "W" AND amount <= balance
      balance = balance - amount
   ELSE
      PRINT "Invalid transaction"
PRINT balance
```

STOP

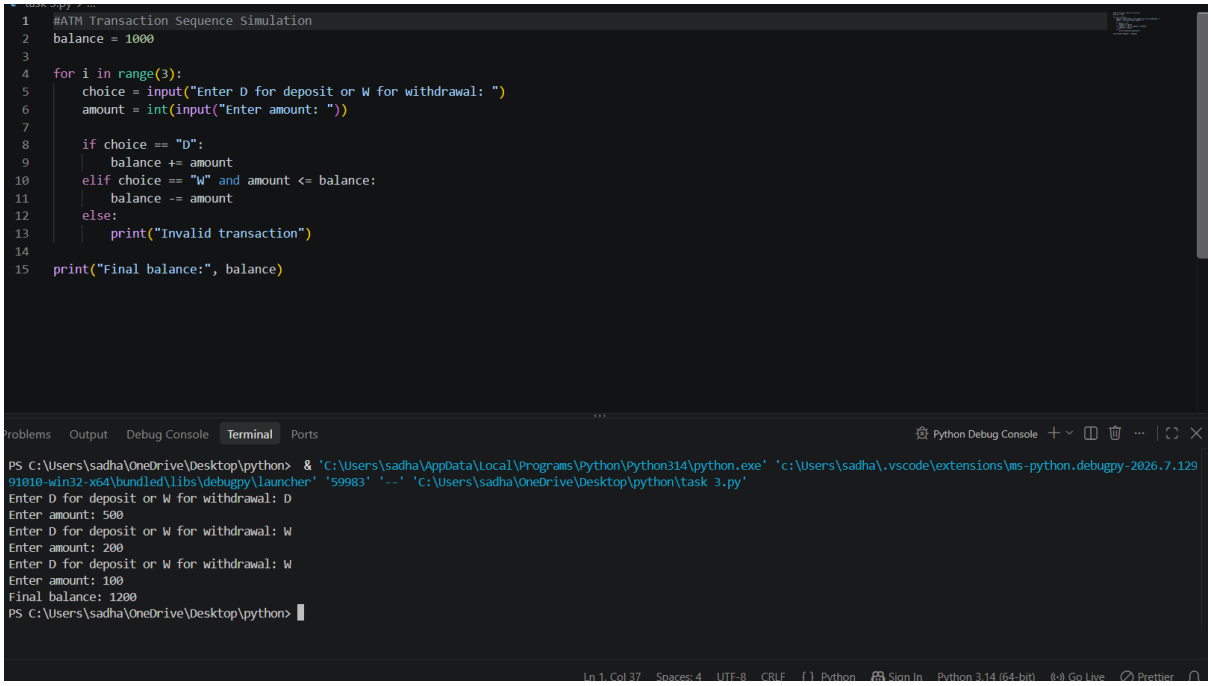
7. Python Source Code

```
balance = 1000
for i in range(3):
 choice = input("Enter D for deposit or W for withdrawal: ")
amount = int(input("Enter amount: "))
  if amount <= 0:
     print("Invalid amount")
 elif choice == "D":
      balance += amount
 elif choice == "W" and amount <= balance:
      balance -= amount
  else:
      print("Invalid transaction")
print("Final balance:", balance)
```

8. Test Cases

Transaction	Amount	Expected result
D	500	Balance becomes ₹1500
W	200	Balance decreases by ₹200
W	2000	Invalid transaction
D	-100	Invalid amount

9. Output



10. Conclusion

The program simulates three ATM transactions and displays the final account balance.

Program 4: Student Performance Trend Analysis

1. Problem Statement

To compare a student's marks in two examinations and identify the performance trend.

2. Objectives

- To compare two values.
- To use if-elif-else statements.

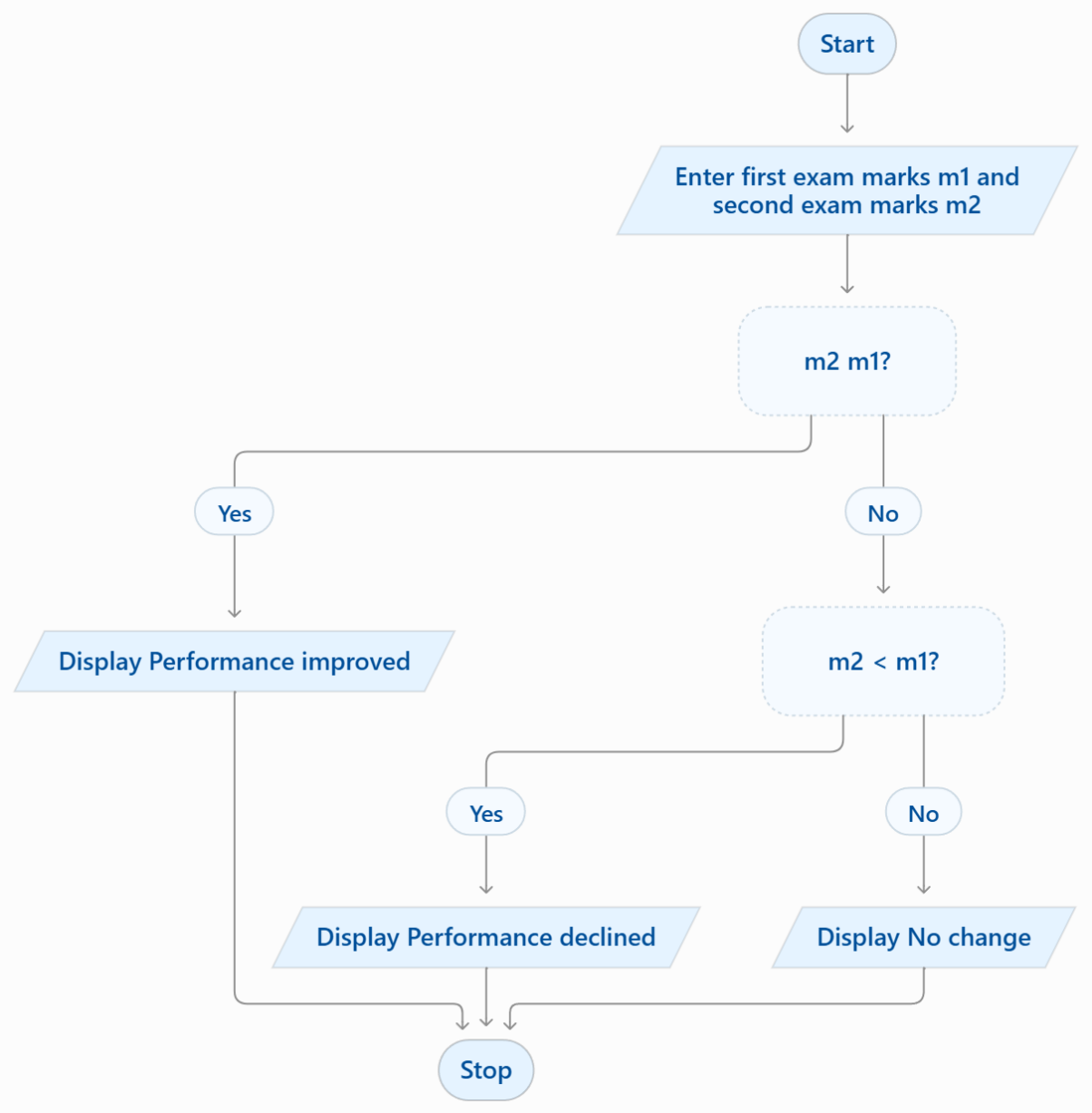
3. Assumptions

- Marks are entered as integers.
- Marks are between 0 and 100.
- Both examinations use the same maximum marks.

4. Algorithm

1. Start.
2. Input first and second examination marks.
3. Compare the marks.
4. If the second mark is higher, display improved.
5. If lower, display declined.
6. Otherwise, display no change.
7. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT m1, m2
IF m1 < 0 OR m1 > 100 OR m2 < 0 OR m2 > 100
   PRINT "Invalid marks"
ELSE IF m2 > m1
   PRINT "Performance improved"
ELSE IF m2 < m1
   PRINT "Performance declined"
ELSE
   PRINT "No change"
STOP
```

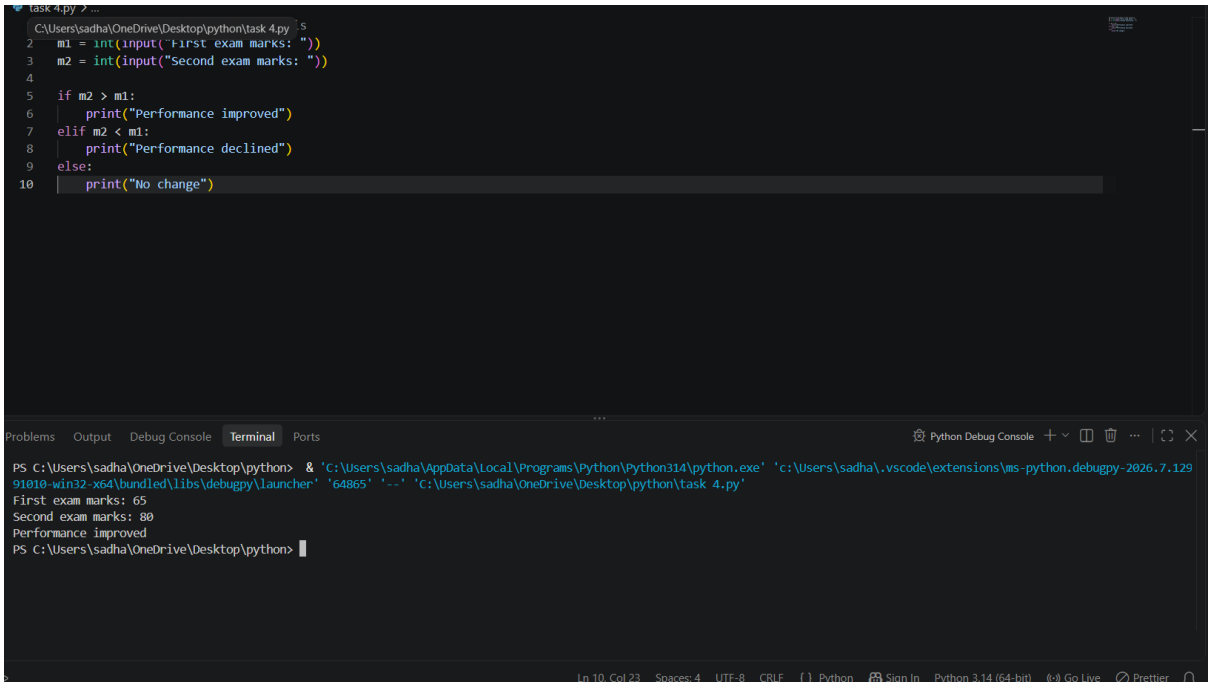
7. Python Source Code

```
m1 = int(input("First exam marks: "))
m2 = int(input("Second exam marks: "))
if m2 > m1:
  print("Performance improved")
elif m2 < m1:
  print("Performance declined")
else:
 print("No change")
```

8. Test Cases

First mark	Second mark	Expected output
60	80	Performance improved
90	70	Performance declined
75	75	No change
110	80	Invalid marks

9.Output



10. Conclusion

The program compares examination marks and identifies whether student performance improved, declined, or remained unchanged.

Program 5: Fuel Consumption and Route Comparison

1. Problem Statement

To calculate the fuel required and travel cost for a given distance.

2. Objectives

- To perform arithmetic calculations.
- To understand floating-point values and input validation.

3. Assumptions

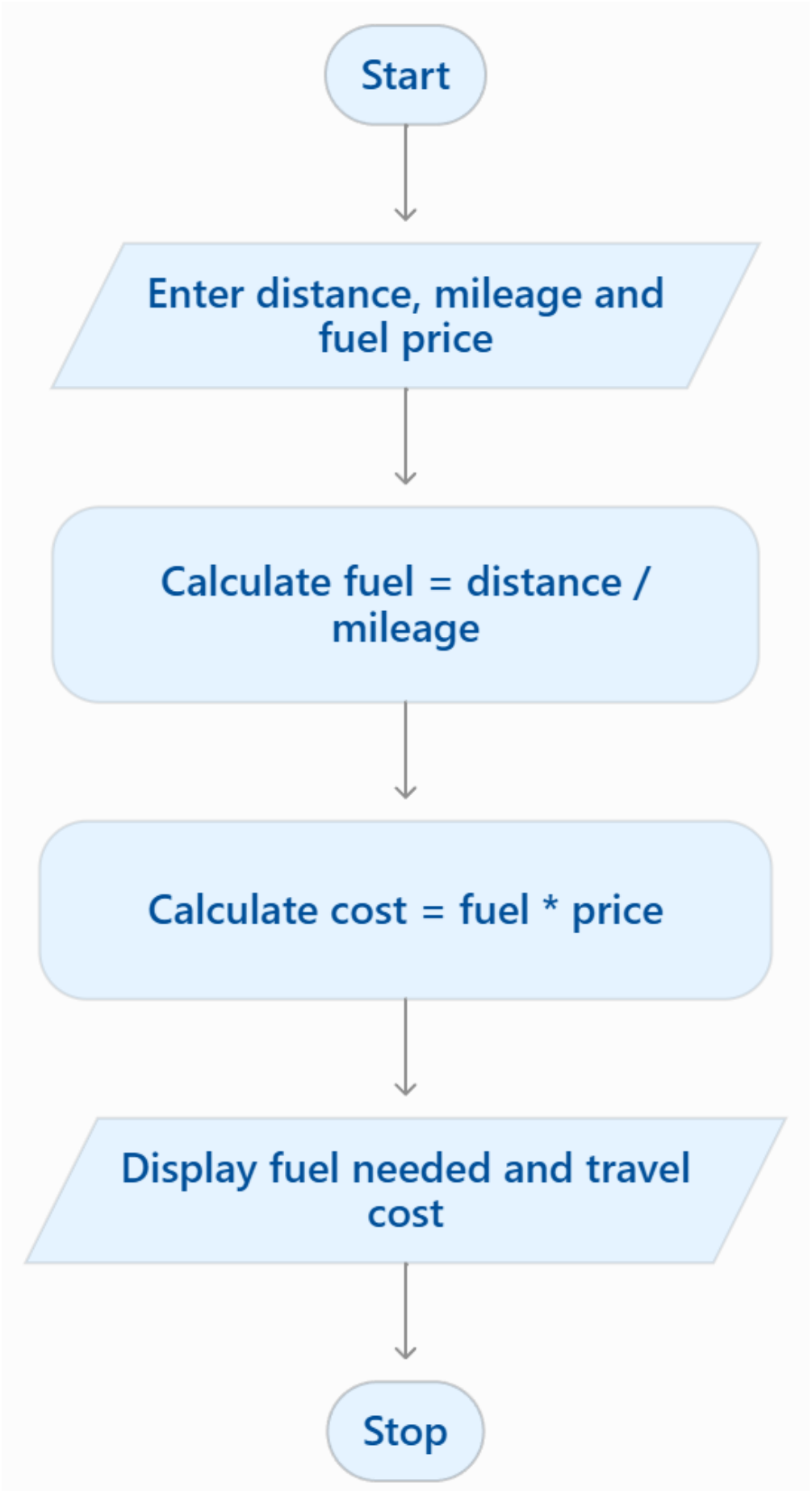
- Distance is measured in kilometres.
- Mileage is measured in kilometres per litre.
- Fuel price is entered per litre.

- Distance cannot be negative, and mileage must be greater than zero.

4. Algorithm

1. Start.
2. Input distance, mileage, and fuel price.
3. Validate the inputs.
4. Calculate fuel required as distance divided by mileage.
5. Calculate cost as fuel required multiplied by fuel price.
6. Display fuel required and travel cost.
7. Stop.

5. Flowchart


Start
Enter distance, mileage and fuel price
Calculate fuel = distance / mileage
Calculate cost = fuel * price
Display fuel needed and travel cost
Stop

6. Pseudocode

```
START
INPUT d, m, p
IF d < 0 OR m <= 0 OR p < 0
   PRINT "Invalid input"
ELSE
   fuel = d / m
   cost = fuel * p
   PRINT fuel, cost
STOP
```

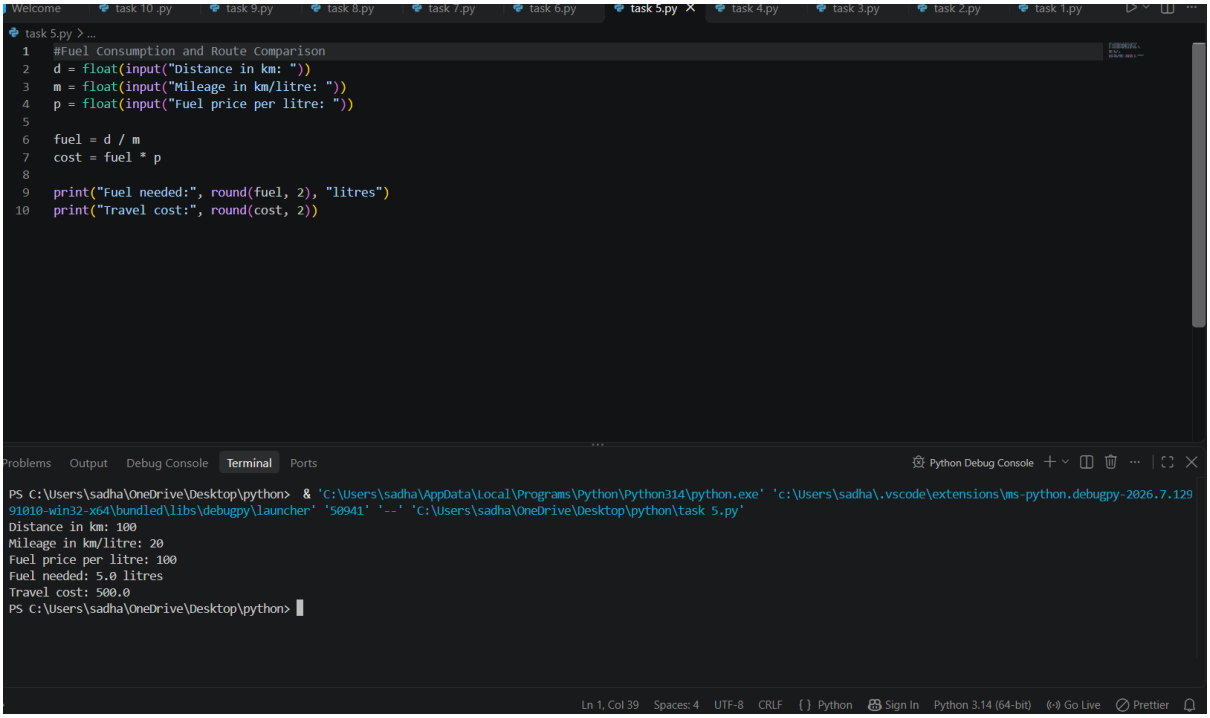
7. Python Source Code

```
d = float(input("Distance in km: "))
m = float(input("Mileage in km/litre: "))
p = float(input("Fuel price per litre: "))
  fuel = d / m
 cost = fuel * p
  print("Fuel needed:", round(fuel, 2), "litres")
print("Travel cost:", round(cost, 2))
```

8. Test Cases

Distance	Mileage	Price	Expected result
100	20	100	5 litres; cost 500
50	10	100	5 litres; cost 500
100	0	100	Invalid input
-10	20	100	Invalid input

9.Output



10. Conclusion

The program calculates the fuel required and estimated travel cost using arithmetic operations.

Program 6: Repeated Transaction Detection

1. Problem Statement

To identify repeated transaction IDs from a list entered by the user.

2. Objectives

- To work with lists and strings.
- To identify duplicate values.
- To use loops and conditions.

3. Assumptions

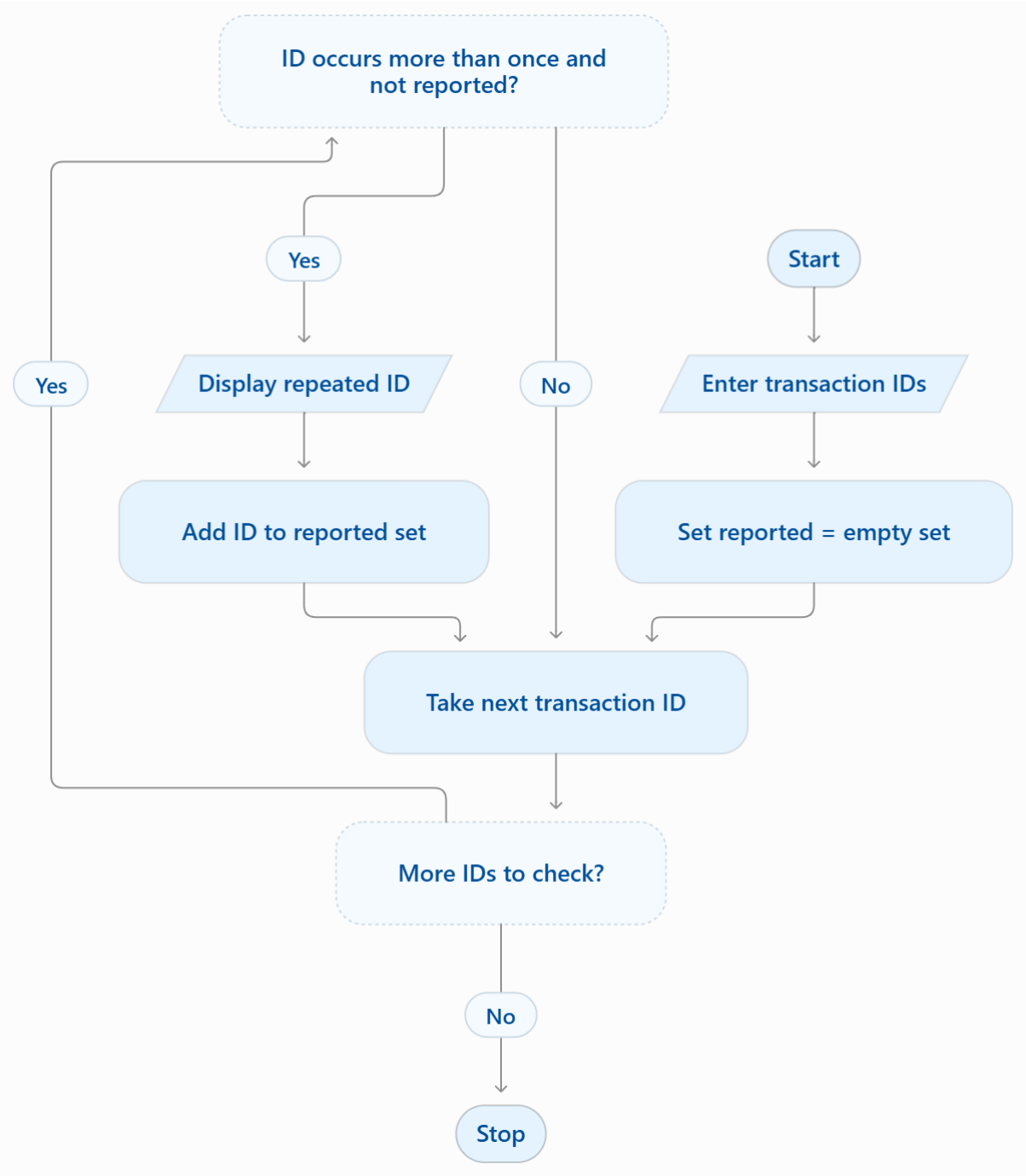
- Transaction IDs are separated by spaces.
- IDs are treated as text.

- Each repeated ID should be displayed only once.

4. Algorithm

1. Start.
2. Input transaction IDs separated by spaces.
3. Convert the input into a list.
4. Create an empty set to store reported IDs.
5. Check each ID in the list.
6. If an ID occurs more than once and has not been reported, display it and record it.
7. Stop.

5. Flowchart



6. Pseudocode

START

INPUT transaction IDs

SPLIT IDs into a list

CREATE empty set reported

FOR each ID in list

IF count(ID) > 1 AND ID not in reported

PRINT "Repeated ID:", ID

ADD ID to reported

STOP

7. Python Source Code

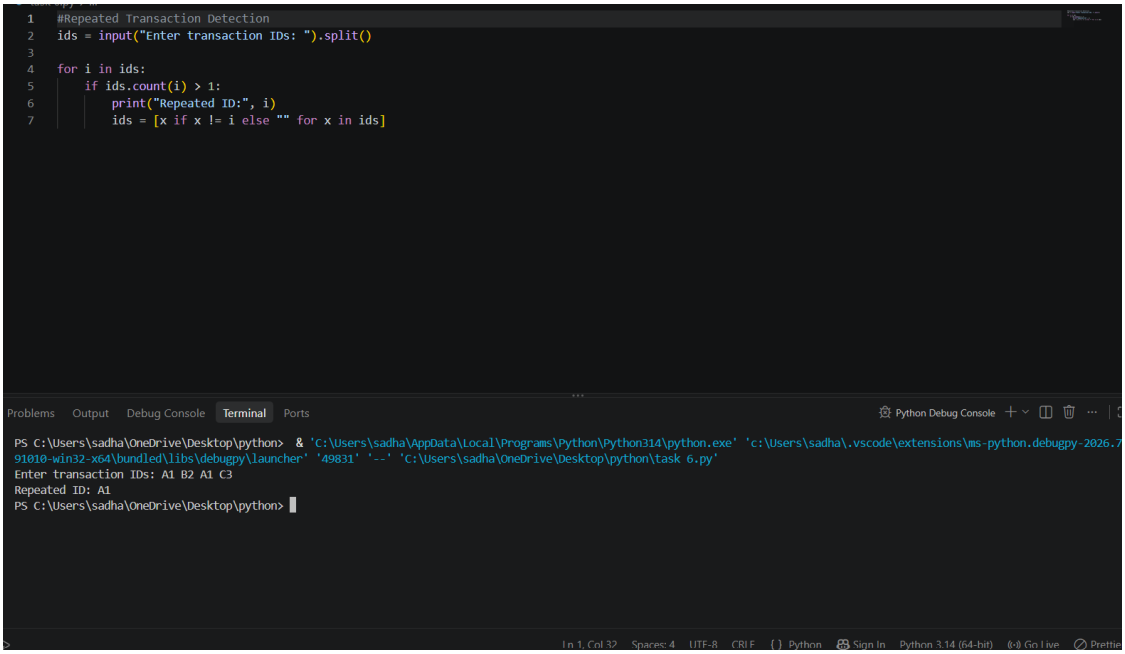
```
ids = input("Enter transaction IDs: ").split()


for i in ids:
  if ids.count(i) > 1:
    print("Repeated ID:", i)
    ids = [x if x != i else "" for x in ids]
```

8. Test Cases

Test Case	Input: Transaction IDs	Expected Output
1	T1 T2 T1 T3	Repeated ID: T1
2	A1 B2 C3	No repeated IDs displayed
3	X1 X1 X1	Repeated ID: X1
4	P1 P2 P2 P3 P3	Repeated ID: P2, Repeated ID:P3
5	A1 A2 A3 A2 A1	Repeated ID: A1, Repeated ID:A2

9.Output



10. Conclusion

The program detects duplicate transaction IDs and displays each repeated ID only once.

Program 7: Student Email and Identifier Validation

1. Problem Statement

To perform a basic check on a student's email address using Python string operations.

2. Objectives

- To understand string operations.
- To use conditional statements for basic validation.

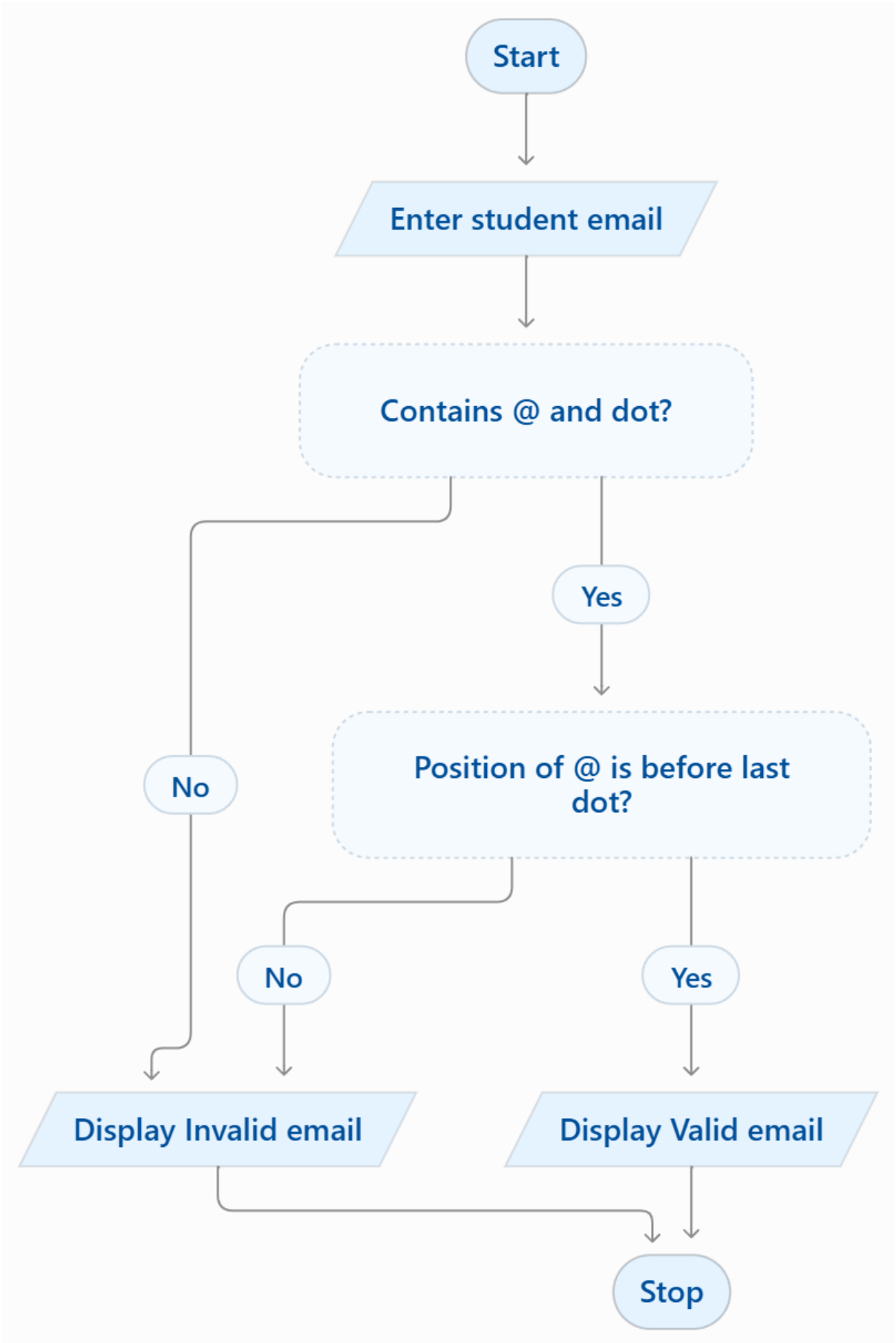
3. Assumptions

- The email address is entered as a string.
- A valid result means only that the basic checks in this program are satisfied.
- This program does not perform complete email verification.

4. Algorithm

1. Start.
2. Input the student's email address.
3. Check whether @ and . are present.
4. Check whether the first @ occurs before the last ..
5. Display Valid email if the conditions are satisfied.
6. Otherwise, display Invalid email.
7. Stop.

5. Flowchart


Start
Enter student email
Contains @ and dot?
Yes
No
Position of @ is before last dot?
No
Yes
Display Invalid email
Display Valid email
Stop

6. Pseudocode

START

INPUT email

IF "@" in email AND "." in email

  IF position of "@" < position of last "."

    PRINT "Valid email"

  ELSE

    PRINT "Invalid email"

ELSE

  PRINT "Invalid email"

STOP

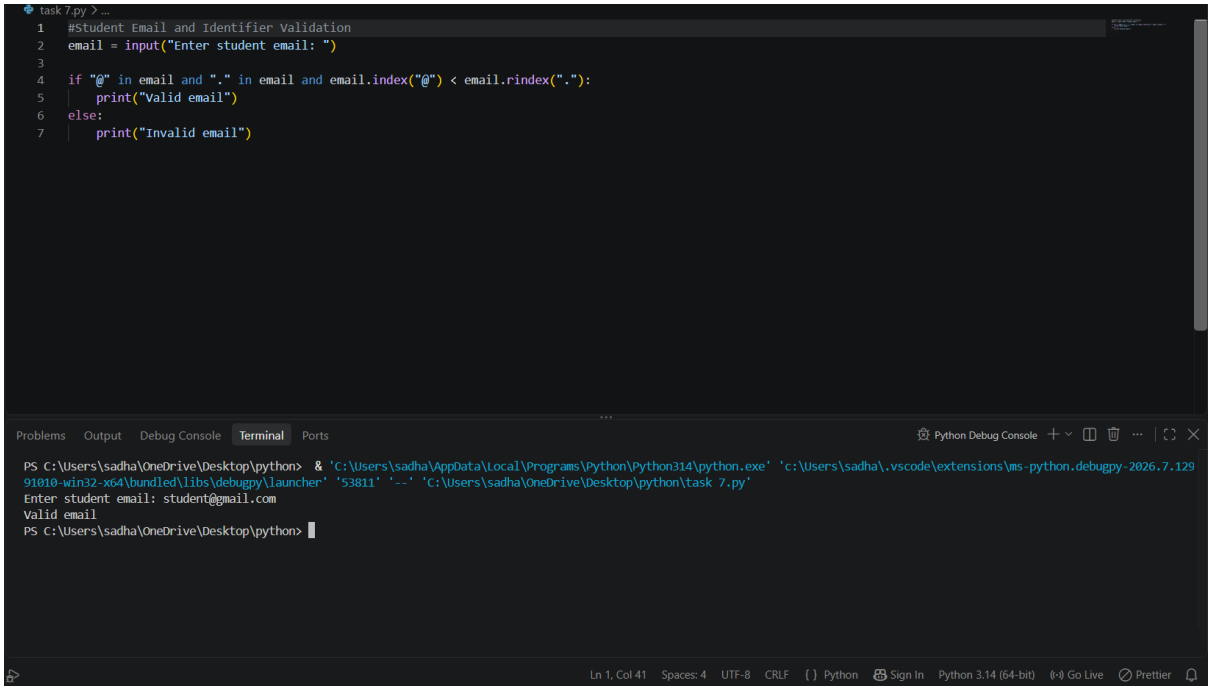
7. Python Source Code

```
email = input("Enter student email: ")
if "@" in email and "." in email and email.index("@") < email.rindex("."):
 print("Valid email")
else:
  print("Invalid email")
```

8. Test Cases

Input	**Expected output**
student@example.com	Valid email
studentexample.com	Invalid email
student@domain	Invalid email
abc.com@	Invalid email

9. Output



10. Conclusion

The program performs basic email-format checks using Python string operations. It does not guarantee that an email address actually exists.

Program 8: Student Marks Matrix Analysis

1. Problem Statement

To calculate and display the total marks of each student using a nested list.

2. Objectives

- To understand nested lists.
- To use for loops and the sum() function.
- To calculate student totals.

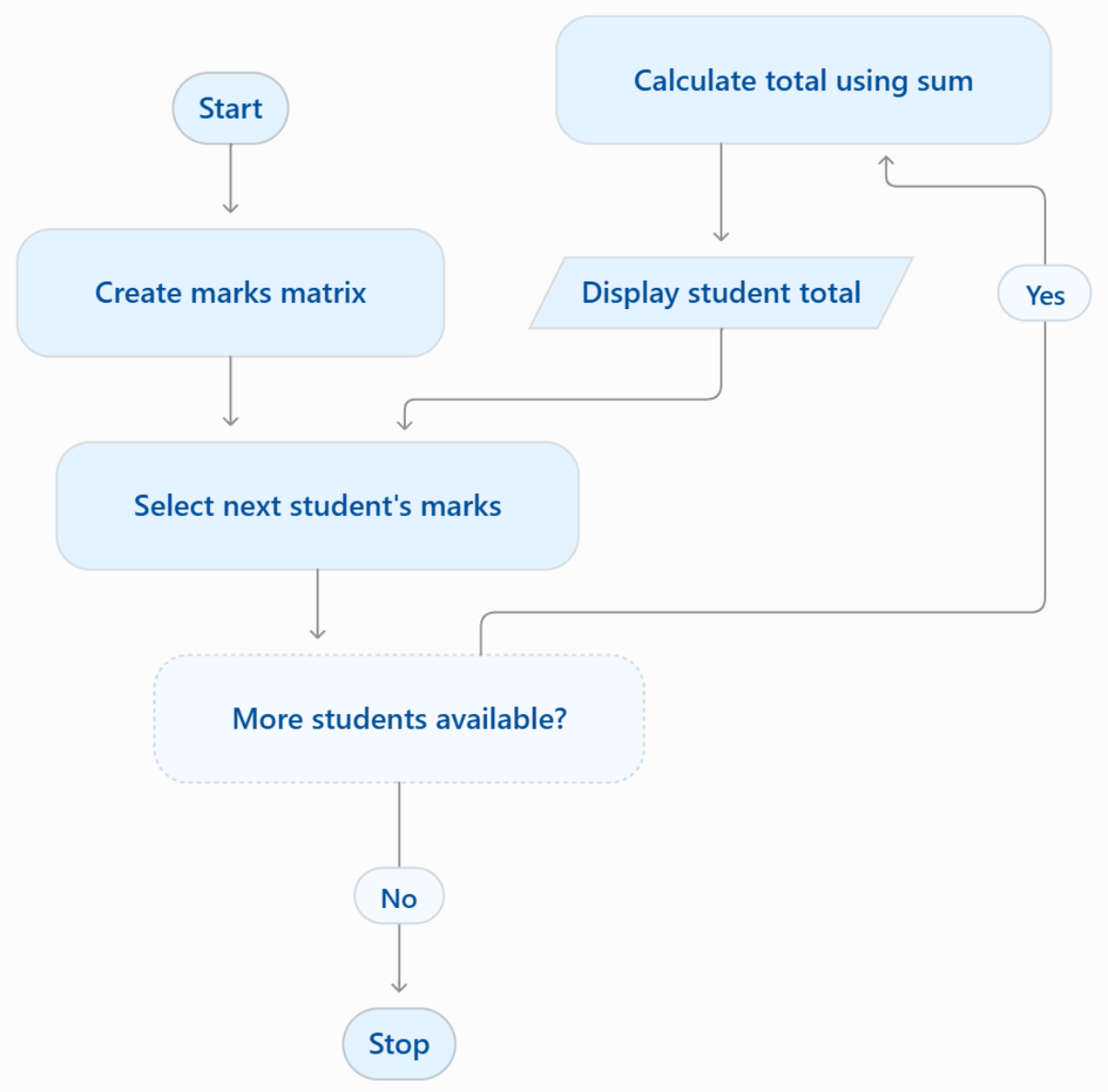
3. Assumptions

- Marks are stored for three students.
- Each student has marks in three subjects.
- The marks are predefined in the program.

4. Algorithm

1. Start.
2. Create a nested list containing student marks.
3. Take each student's marks list.
4. Calculate the total using sum().
5. Display the total.
6. Repeat for all students.
7. Stop.

5. Flowchart



6. Pseudocode

START

marks = [[80, 70, 90], [60, 75, 85], [90, 95, 88]]

FOR each student in marks

  total = SUM(student)

  PRINT total

STOP

7. Python Source Code

```
marks = [[80, 70, 90], [60, 75, 85], [90, 95, 88]]

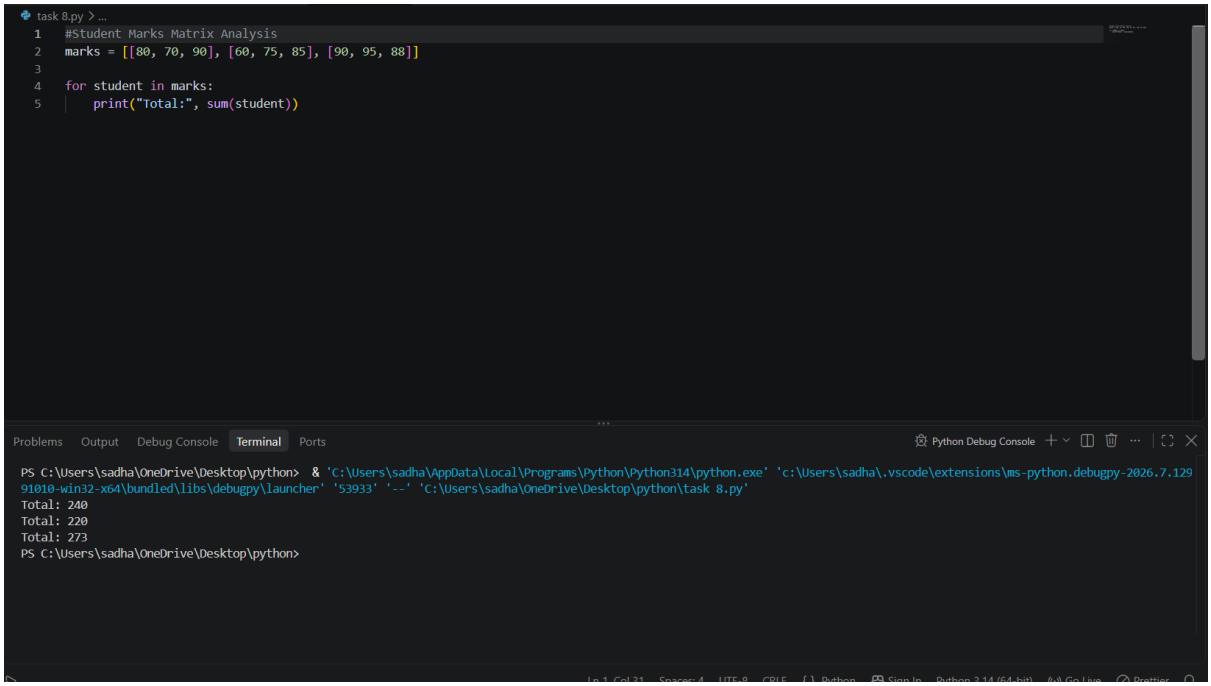
for student in marks:

    print("Total:", sum(student))
```

8. Test Cases

Student marks	Expected total
80, 70, 90	240
60, 75, 85	220
90, 95, 88	273

9. Output



10. Conclusion

The program uses a nested list and a loop to calculate and display each student's total marks.

Program 9: Tuple-Based Product Catalogue Audit

1. Problem Statement

To identify products that need restocking by checking their stock quantities using tuples.

2. Objectives

- To understand tuples and lists.
- To access tuple elements using unpacking.
- To use conditional statements to identify low-stock products.

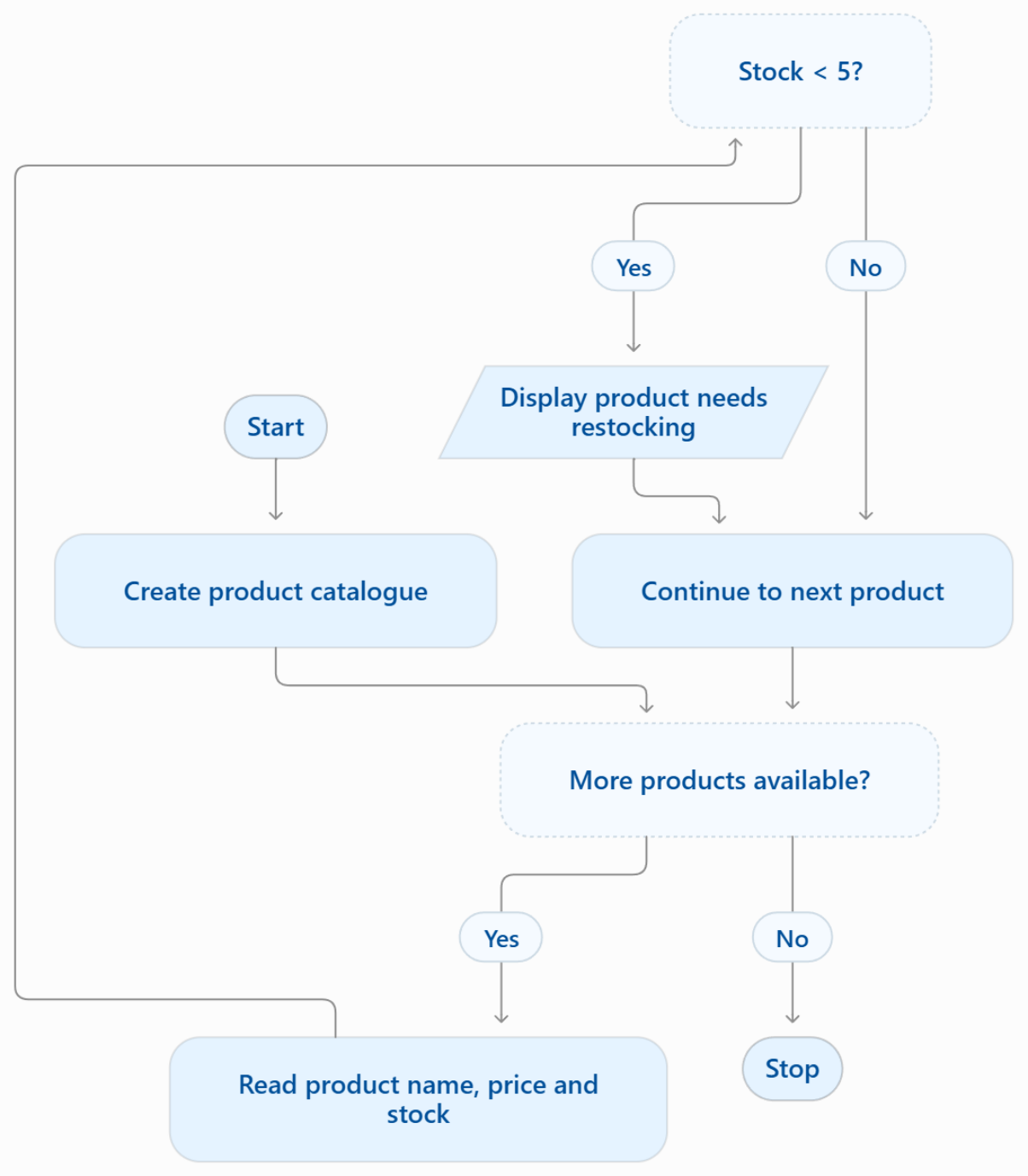
3. Assumptions

- Each product contains its name, price, and stock quantity.
- A product needs restocking when its stock is less than 5.
- Product details are predefined.

4. Algorithm

1. Start.
2. Create a list of product tuples.
3. Read each product's name, price, and stock.
4. Check whether the stock is less than 5.
5. If true, display the product name.
6. Repeat for all products.
7. Stop.

5. Flowchart



6. Pseudocode

START

products = product catalogue

FOR each product in products

READ name, price, stock

IF stock < 5

DISPLAY name, "needs restocking"

STOP

7. Python Source Code

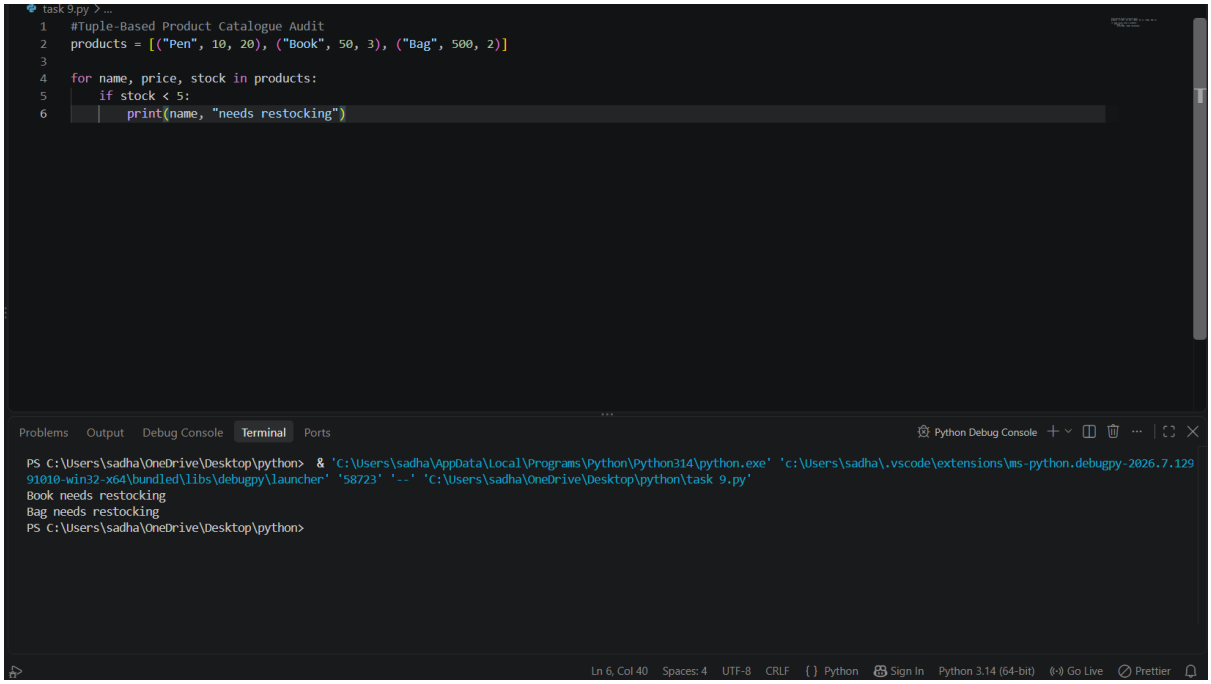
```
products = [
  ("Pen", 10, 20),
  ("Book", 50, 3),
  ("Bag", 500, 2)
]

for name, price, stock in products:
  if stock < 5:
    print(name, "needs restocking")
```

8. Test Cases

Product	Price	Stock	Expected result
Pen	10	20	No restocking needed
Book	50	3	Needs restocking
Bag	500	2	Needs restocking

9. Sample Output



10. Conclusion

The program uses tuples, a for loop, and an if condition to identify products that need restocking

10: Food Waste Monitoring System

1. Problem Statement

To calculate the number of leftover meals and the food wastage percentage using the number of meals prepared and served.

2. Objectives

- **To accept meals prepared and served.**
- **To calculate leftover meals.**
- **To calculate food wastage percentage.**

3. Assumptions

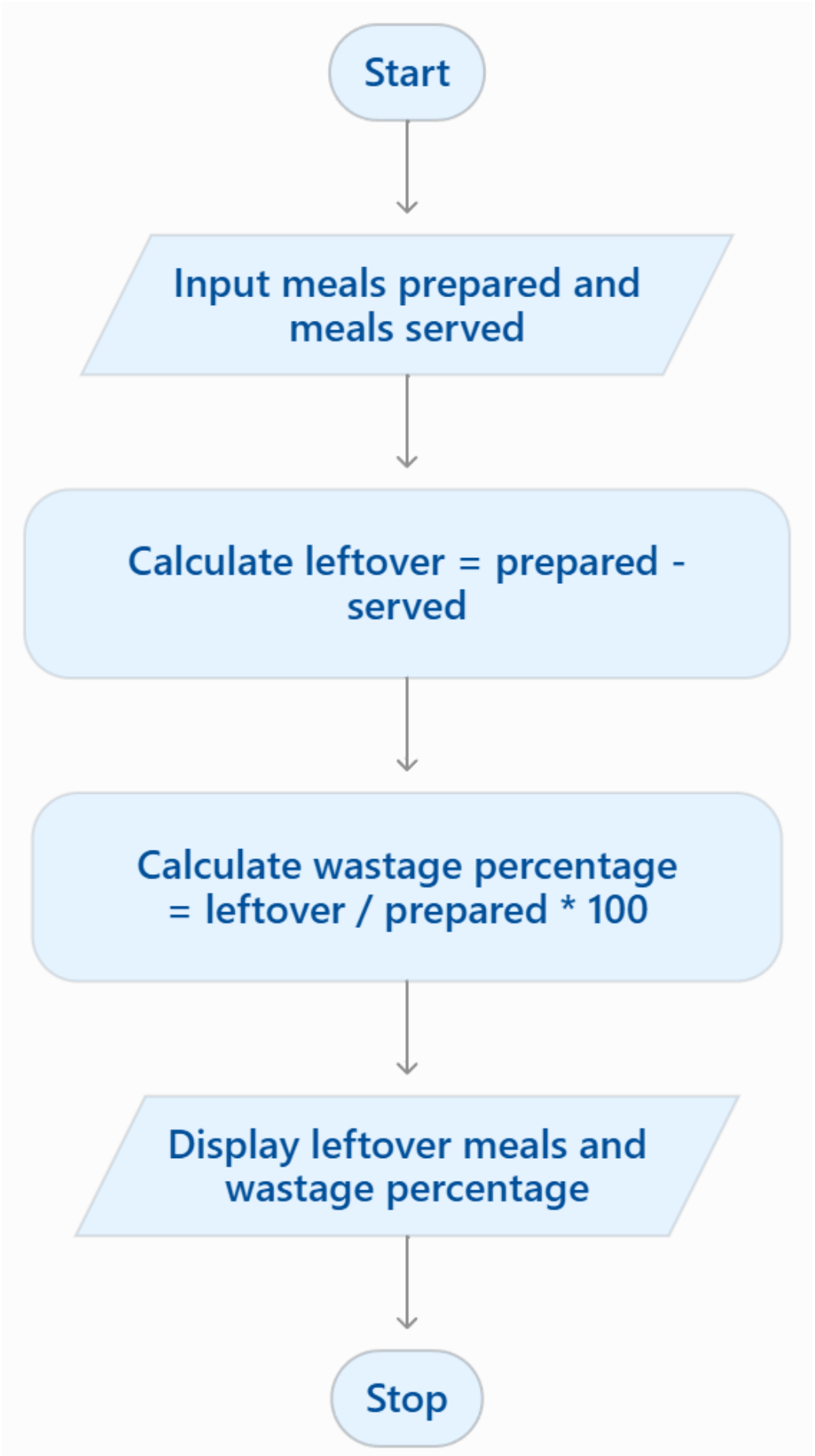
- **Meals prepared must be greater than zero.**
- **Meals served must be between zero and meals prepared.**

- **Leftover meals are calculated by subtracting meals served from meals prepared.**

4. Algorithm

1. **Start.**
2. **Input meals prepared and meals served.**
3. **Check whether the inputs are valid.**
4. **Calculate leftover meals = meals prepared − meals served.**
5. **Calculate wastage percentage = leftover meals / meals prepared × 100.**
6. **Display leftover meals and wastage percentage.**
7. **Stop.**

5. Flowchart


Start
Input meals prepared and meals served
Calculate leftover = prepared - served
Calculate wastage percentage = leftover / prepared * 100
Display leftover meals and wastage percentage
Stop

6. Pseudocode

```
START
INPUT prepared, served
IF prepared <= 0 OR served < 0 OR served > prepared
   DISPLAY "Invalid input"
ELSE
   leftover = prepared - served
   waste = leftover / prepared * 100
   DISPLAY leftover
   DISPLAY waste
STOP
```

7. Python Source Code

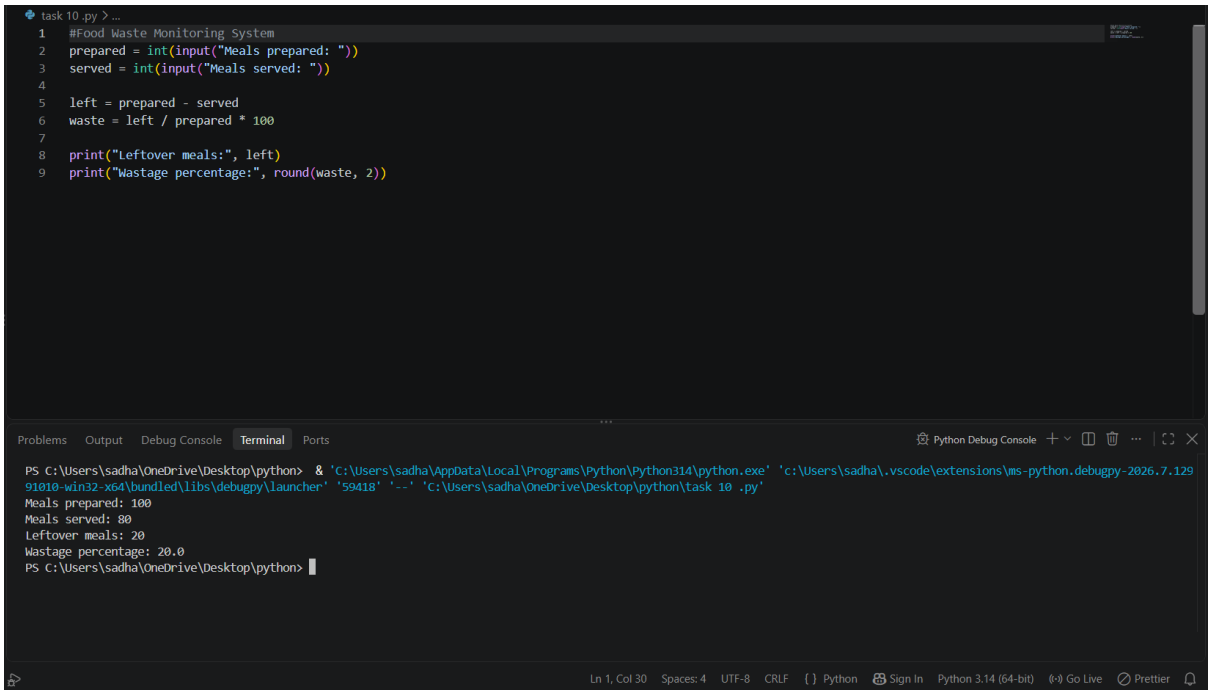
```
prepared = int(input("Meals prepared: "))
served = int(input("Meals served: "))
if prepared <= 0 or served < 0 or served > prepared:
 print("Invalid input")
else:
 left = prepared - served
 waste = left / prepared * 100
print("Leftover meals:", left)
print("Wastage percentage:", round(waste, 2), "%")
```

8. Test Cases

Prepared	Served	Expected leftover	Wastage
100	80	20	20%
50	50	0	0%
40	30	10	25%

0 0 Invalid input Invalid input

9.Sample Output



10. Conclusion

The program calculates leftover meals and food wastage percentage using arithmetic operations and conditional statements.